

Day 1 Lab Sheet: Build the Notes Board Pages

Goal: By the end of this lab you will have a styled Notes Board home page and an About page, built by you, in your own folder.

Time: about 75 minutes of building, plus the quiz.

What you need: A Windows computer with VS Code and a web browser. You do not need to install anything today.

How to use this sheet: Do the steps in order. Type the code yourself. Typing helps you learn much more than copying. After each part there is a **Check** box. If your page does not match the check, look at the Stuck Sheet, or ask your instructor.

If you fall behind, you can start from a later point:

- The `starter` folder gives you an empty page skeleton.
- The `done` folder is the finished result for today.

Part 0: Make file extensions visible (2 minutes)

Windows can hide the end of a file name, such as `.html`. This causes many mistakes, so we turn it on first.

1. Open **File Explorer** (the yellow folder icon).
2. Click **View** in the top menu.
3. Choose **Show**, then tick **File name extensions**.

Check: A file called `report.txt` now shows `.txt` at the end of its name.

Part 1: Create your project folder (5 minutes)

1. Open **Visual Studio Code**.
2. Click **File**, then **Open Folder**.
3. Go to your **Documents** folder. Click **New folder**, name it `Workshop`, then open it and click **Select Folder**.
4. If VS Code asks "Do you trust the authors of the files in this folder?", choose **Yes, I trust the authors**.
5. In the **Explorer** panel on the left, click the **New Folder** icon, type `notes-board` and press Enter.

- Click on `notes-board` to select it. Click the **New Folder** icon again, type `css` and press Enter.

Turn on **Auto Save**, so you never forget to save: click **File** and tick **Auto Save**.

Check: The Explorer panel shows `Workshop`, with `notes-board` inside it, and `css` inside that.

Rule for today: use only lowercase letters in file and folder names, and use `-` instead of spaces.

Part 2: Create the page skeleton (5 minutes)

- Click on the `notes-board` folder. Click the **New File** icon. Type `index.html` and press Enter.
- The file opens in the editor. On the first line, type `!` and then press the **Tab** key. VS Code writes the basic structure of a web page for you. This shortcut is called **Emmet**.
- Find the line `<title>Document</title>` and change it to:

```
<title>My Notes Board</title>
```

- Inside `<head>`, below the title, add a line that links your style file (we will create the file in Part 7):

```
<link rel="stylesheet" href="css/style.css">
```

Your file should now look like this:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Notes Board</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>

</body>
</html>
```

What did we do? The `<head>` holds information about the page. The `<body>` will hold everything the user sees.

Check: Your file has `<head>` and `<body>`, and the title says `My Notes Board`.

Part 3: The header and the main area (10 minutes)

Type this code **between** `<body>` and `</body>`:

```
<header class="site-header">
  <h1>My Notes Board</h1>
  <nav>
    <a href="index.html">Home</a>
    <a href="about.html">About</a>
  </nav>
</header>

<main class="container">

</main>
```

What did we do?

- `<header>` is the top part of the page.
- `<h1>` is the main heading. There should be only one on a page.
- `<nav>` holds the links. The `href` attribute says where each link goes. The About page does not exist yet. We create it in Part 9.
- `<main>` holds the main content of the page.
- `class="..."` gives a name to an element, so that CSS can find it later.

Check: Save the file. You do not see the page yet, because we have not opened it in the browser. That comes in Part 6.

Part 4: The "Add a note" form (10 minutes)

Type this code **inside** `<main>`, between `<main class="container">` and `</main>`:

```

<section class="card">
  <h2>Add a note</h2>
  <form id="note-form">
    <label for="note-title">Title</label>
    <input type="text" id="note-title" name="title" placeholder="Example: Buy a
notebook" maxlength="60">

    <label for="note-text">Note</label>
    <textarea id="note-text" name="text" rows="4" placeholder="Write your note
here" maxlength="300"></textarea>

    <button type="submit">Add note</button>
  </form>
</section>

```

What did we do?

- `<section>` groups a part of the page.
- `<form>` is a form. Its `id` is a unique name.
- `<label for="note-title">` is the name of a field. The `for` value matches the `id` of the field. Because of this, clicking the label puts the cursor in the field.
- `<input type="text">` is a one-line box. `placeholder` is the grey hint text. `maxlength` limits the letters.
- `<textarea>` is a bigger box for many lines.
- `<button type="submit">` sends the form.

Part 5: The list of notes (10 minutes)

Type this code **inside** `<main>`, **after** the closing `</section>` of the form:

```

<section>
  <h2>All notes</h2>
  <div id="notes-list">
    <article class="note">
      <h3>Welcome</h3>
      <p>This is a sample note. On Day 2, you will make this page add real notes.
</p>
    </article>
    <article class="note">
      <h3>Study plan</h3>
      <p>Learn HTML on Monday, CSS on Tuesday, and JavaScript on Wednesday.</p>
    </article>
    <article class="note">
      <h3>Idea</h3>
      <p>Use this app every day to keep my class notes in one place.</p>
    </article>
  </div>
</section>

```

Now add a footer. Type it **after** the closing `</main>` and **before** `</body>` :

```

<footer class="site-footer">
  <p>Built in the Web Technology workshop at Mandsaur University.</p>
</footer>

```

What did we do? Each note is an `<article>` with a title (`<h3>`) and a paragraph (`<p>`). On Day 2, JavaScript will create these automatically. Today we write three by hand, so that we can design them.

Part 6: See your page in the browser (5 minutes)

1. Open **File Explorer** and go to `Documents\Workshop\notes-board` .
2. **Double-click** `index.html` . It opens in your browser.

You will see a plain page with no colours. This is correct. HTML only gives structure.

3. Click the **Add note** button. The page reloads. This is normal today, because the form is not connected to anything yet.
4. Go back to VS Code, change a word in one of the notes, and save. Then go to the browser and press **F5** to refresh. You see your change. This is your **edit, save, refresh** loop. You will use it all week.

Check: You can see the heading, the form, and three notes. The Home link works. The About link shows an error for now. That is fine.

Part 7: Create the stylesheet (5 minutes)

1. In VS Code, click the `css` folder. Click **New File**, type `style.css`, and press Enter.
2. Type this at the top of the file. These are **CSS variables**. They store our colours in one place:

```
/* Notes Board styles (Day 1)
   Tip: change the colours in :root to make the app your own. */

:root {
  --page-bg: #f3f4f6;
  --card-bg: #ffffff;
  --text: #1f2937;
  --muted: #6b7280;
  --accent: #2563eb;
  --accent-dark: #1d4ed8;
  --border: #d1d5db;
  --radius: 10px;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Segoe UI", Arial, sans-serif;
  background: var(--page-bg);
  color: var(--text);
  line-height: 1.5;
}
```

3. Save, go to the browser, and press **F5**.

Check: The page background is now light grey, and the font changed.

What did we do?

- `:root` is the top of the page. Variables written here can be used everywhere.

- `*` means "every element". `box-sizing: border-box` makes the width of a box include its padding. This makes layouts easier.
- `body` styles the whole page. `var(--page-bg)` uses the variable we made.

Part 8: Style the header, the layout and the cards (15 minutes)

8a. The header and navigation

Add this to the **end** of `style.css` :

```
/* Header and navigation */
.site-header {
  background: var(--accent);
  color: #ffffff;
  padding: 16px 24px;
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  justify-content: space-between;
  gap: 8px;
}

.site-header h1 {
  margin: 0;
  font-size: 1.5rem;
}

.site-header nav a {
  color: #ffffff;
  text-decoration: none;
  margin-left: 16px;
  font-weight: 600;
}

.site-header nav a:hover {
  text-decoration: underline;
}
```

Save, refresh, and look at the header.

- `.site-header` with a dot means "all elements with `class="site-header"`".

- `display: flex` puts the title and the links on one line. `justify-content: space-between` pushes them to the two ends.
- `:hover` is a style that applies only when the mouse is over the element.

8b. The layout and the cards

Add this to the **end** of `style.css` :

```
/* Page layout */
.container {
  max-width: 720px;
  margin: 24px auto;
  padding: 0 16px;
}

.card {
  background: var(--card-bg);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 16px 20px;
  margin-bottom: 24px;
}

h2 {
  margin-top: 0;
}
```

Save, refresh, and look at the page.

- `max-width: 720px` stops the page from becoming too wide.
- `margin: 24px auto` means 24 pixels above and below, and `auto` on the left and right centres the box.
- `border-radius` makes the corners round.

8c. Copy the remaining styles

The rest of the CSS is for the form, the notes, the About page and the footer. It is long, and the ideas are the same as before. So we copy it:

1. In VS Code, open the `starter` folder from your instructor and open the file `css/extra-styles.css`. Your instructor will tell you where this folder is.
2. Press **Ctrl + A** to select everything, then **Ctrl + C** to copy.

3. Go back to your `style.css`, go to the very end, press **Enter** twice, and press **Ctrl + V** to paste.
4. Save, and refresh the browser.

Check: Your page looks like a clean app: a blue header, a white form card, and three notes with a blue line on the left.

Try it: Change `--accent: #2563eb;` to another colour, for example `#16a34a` (green). Save and refresh. The whole app changes colour. This is the power of CSS variables.

Part 9: Create the About page (10 minutes)

1. Click the `notes-board` folder. Click **New File** and type `about.html`.
2. Type `!` and press **Tab**, as in Part 2. Change the title to `About - My Notes Board`, and add the same `<link>` line for the stylesheet.
3. Copy the `<header>` block from `index.html` into the `<body>` of `about.html`. (Select it, press **Ctrl + C**, and paste it with **Ctrl + V**.)
4. After the header, add this code. **Replace YOUR NAME and YOUR ROLL NUMBER with your own details.**

```

<main class="container">
  <section class="card">
    <h2>About this project</h2>
    <p>
      This Notes Board was built during a 5-day hands-on Web Technology
      workshop. It shows how the frontend, the backend, the database and
      hosting work together in one small app.
    </p>
  </section>

  <section class="card">
    <h2>Built by</h2>
    <dl class="details">
      <dt>Name</dt>
      <dd>YOUR NAME</dd>
      <dt>Roll number</dt>
      <dd>YOUR ROLL NUMBER</dd>
      <dt>Class</dt>
      <dd>BCA Second Year</dd>
      <dt>College</dt>
      <dd>Mandsaur University</dd>
    </dl>
    <p class="hint">
      You choose what to show here. Delete any line you do not want to share.
    </p>
  </section>

  <section class="card">
    <h2>Guided by</h2>
    <p>
      Instructor: Rahul Dhangar.
      <a href="https://github.com/rahuldhangar" target="_blank" rel="noopener
noreferrer">GitHub profile</a>
    </p>
  </section>
</main>

```

- Copy the `<footer>` block from `index.html` and paste it after `</main>`.
- Save. Open `about.html` in the browser (double-click it), or click the **About** link on your home page.

Check: The About page has the same header, three cards, and the footer. Clicking **Home** takes you back.

Notes about this page:

- `<dl>` , `<dt>` and `<dd>` make a list of terms and their descriptions.
- **You decide what to show.** If you do not want your roll number on a public page, delete that `<dt>` and its `<dd>` . Your name and class are enough. We encourage you to keep the instructor line, because it tells visitors where you learned to build this.
- `target="_blank"` opens a link in a new tab. `rel="noopener noreferrer"` is a safety setting for links that open new tabs. Always use both together.
- Later, your About page will go online with a public link. Only write what you are happy for everyone to see.

Part 10: Look inside with Developer Tools (5 minutes)

1. Open your home page in the browser. Press **F12**.
2. Click the **Elements** tab. You see the HTML of your page. Click on the small arrows to open and close elements.
3. Click on any element. On the right, you see its CSS. Try to untick one property, and watch the page change. This does not change your file. It only changes what you see, so you can test ideas safely.
4. Press **F12** again to close the tools.

Part 11: Save a backup (3 minutes)

Lab computers can be reset. Always keep a copy of your work.

1. Copy your whole `Workshop` folder (from `Documents`) to a USB drive, or to the place your instructor tells you.
2. If you cannot do this, take a photo of your screen with your phone as a small backup.

The `starter` folder for Day 2 already contains everything from Day 1. So if you lose your files, you can continue tomorrow without any loss.

Part 12: Quiz, then homework

1. Do the 5-question quiz that your instructor shares.

2. Read the **Homework for Day 2** section at the end of the Day 1 Theory. Collect the installers into a folder named `Workshop-Installers` , create your GitHub account at home, and write down your questions.
-

Final checklist

Tick each box before you leave.

`index.html` opens in the browser with a blue header, a form and three notes.

`about.html` shows my details, and the Home and About links work.

`css/style.css` exists, and my colours work.

I made a backup copy of my `Workshop` folder.

I know where to find the installers for tomorrow.

I have written down my questions.

Extra challenges (only if you finish early)

1. Add a fourth sample note to the home page.
2. Change the blue colour to your favourite colour using the CSS variables.
3. Change the font size of the note titles (`.note h3`).
4. Add a link to your GitHub profile on the About page, after you create your account.
5. Make the notes show a different left border colour (`border-left` in the `.note` rule).